

第1章

AI 原生软件工程与环境搭建

让 AI 参与开发，让完整系统在第一周真正运行

AI 原生鸿蒙软件工程实践 | AI Native

本章完成后，你应该能够

理解 AI 原生软件

解释 AI 如何改变需求、设计、编码、测试、运行和演化。

划定信任边界

区分模型建议、系统验证、用户确认与真实业务执行。

管理 AI 开发主体

将任务委派给 AI，并用规格、测试和评审控制质量。

搭建鸿蒙端环境

安装 DevEco Studio、SDK、模拟器或真机，运行 ArkTS 应用。

搭建智能体后端

运行 API、LLM、数据库、缓存、流式接口和工具服务。

完成环境验收

打通客户端 → 后端 → 模型 → 工具 → 数据库的最小链路。

本章的成果不是“软件都装好了”，而是完整开发链能够复现运行

软件开发的稀缺资源正在改变

过去更稀缺

- 手工编写大量代码的时间
- 同时掌握多种框架的能力
- 快速生成测试和文档的成本
- 尝试多个方案的实现费用

现在更稀缺

- 定义正确问题和稳定边界
- 给 AI 足够清晰的上下文与规格
- 验证生成结果是否真的正确
- 管理集成、状态、数据和责任

AI 降低了产生代码的成本，却提高了工程判断、验证和系统整合的价值。

AI 原生软件不是“传统软件加一个聊天框”

开发过程有 AI 主体

需求、设计、前端、后端、测试、数据和评审可以由多个 AI 主体协作完成。

运行系统中也有 AI 主体

Agent 理解目标、生成交互、选择工具，并在业务状态中持续行动。

输出不再完全确定

相同输入可能产生不同结果，需要评测集、护栏、版本和可观测性。

模型不能成为最终权威

权限、业务约束、状态机和副作用执行必须由确定性系统控制。

AI 原生软件同时包含两条 AI 链路

开发时：AI 帮我们构建系统

需求智能体

生成特殊需求、边界和验收条件

开发智能体

生成客户端、后端、数据库与迁移代码

测试与评审智能体

生成测试、攻击样本并检查风险

运行时：AI 帮用户完成任务

对话与需求理解

识别目标、约束、未知与用户纠正

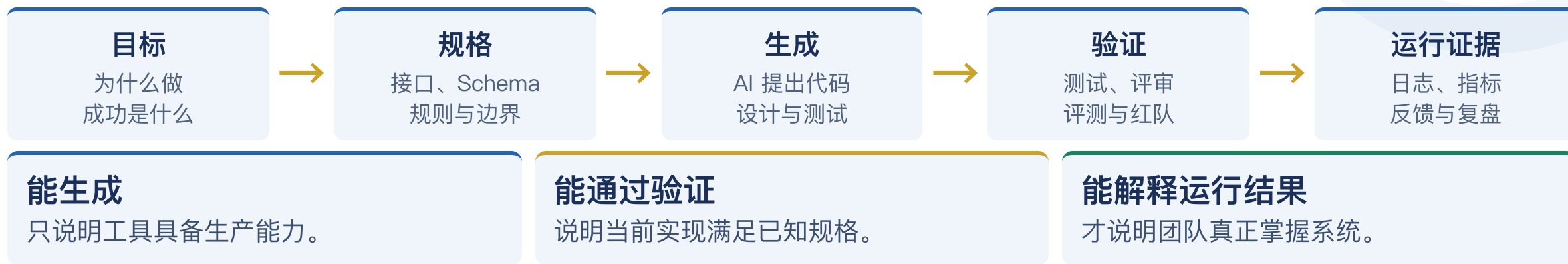
生成式 UI

按任务生成确认卡、比较卡和进度卡

可信工具调用

通过权限和确认完成真实业务操作

AI 原生软件工程是一条证据链



课程不按 AI 生成代码量评分，而按证据链是否完整评分。

用规格约束 AI，而不是靠“再提醒一次”

自然语言目标

说明用户价值、情境、约束和成功证据。

类型与 Schema

约束输入、输出、事件、UI 和工具参数。

业务不变量

定义任何实现都不能违反的状态与权限规则。

示例与反例

帮助 AI 理解边界，而不只是正常路径。

自动化测试

将重要规则变成可重复执行的检查。

评测与红队集

覆盖非确定输出、攻击输入和系统失败。

模型建议、系统验证和真实执行必须分层



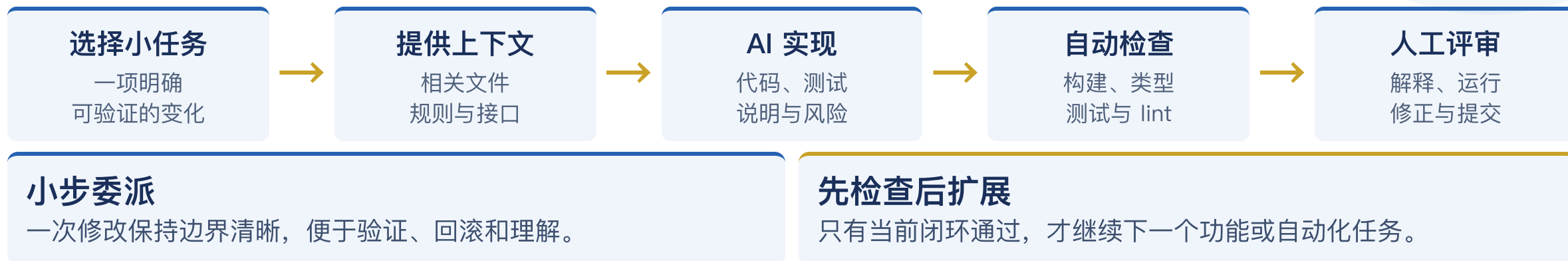
不要让模型直接写数据库、执行任意代码或根据自然语言绕过业务工具。

团队可以组织多个 AI 开发角色

AI 角色	主要任务	必须提供的上下文	人类验收
需求	场景、边界、验收条件	用户证据、业务本体	是否真实且可验证
鸿蒙端	ArkTS、ArkUI、Schema Renderer	组件规范、接口、状态	真机/模拟器运行
后端	API、Agent、工具、数据	契约、权限、不变量	测试与故障恢复
测试	单元、契约、E2E、攻击样本	风险清单、历史缺陷	覆盖关键失败方式
评审	结构、安全、性能、隐私	完整 diff 与架构决策	逐条验证问题

角色可以由不同工具承担，但团队必须统一规格、版本和最终责任

AI 开发的最小闭环



“让 AI 一次生成整个项目”通常只是把风险推迟到集成阶段

课堂练习：审查一段 AI 生成方案

AI 建议

“为了快速实现 AI Native，客户端直接把用户输入发送给大模型。模型返回 ArkTS 代码，客户端动态执行；模型也可以直接访问数据库完成订单操作。”

10 分钟小组讨论

1. 哪些地方把模型建议当成了系统事实？
2. 哪些权限过大？
3. 哪些输出无法验证？
4. 模型、客户端、业务工具和数据库应该怎样分层？
5. 最小安全替代方案是什么？

输出：画出“模型提出—系统验证—用户确认—工具执行”的替代链路

课堂练习参考答案：用确定性安全壳包围模型

客户端直接调用模型

会暴露模型密钥，绕过统一认证、配额、脱敏、版本管理与审计。

动态执行模型生成代码

代码输出无法预先验证，可能越权、访问敏感资源、破坏状态或引入供应链风险。

模型直接访问数据库

违反最小权限，绕过业务状态机、事务、资源级鉴权、用户确认和审计。

自然语言直接驱动行动

含义模糊、不可稳定解析，也无法区分“模型建议”和“已确认业务事实”。

客户端表达目标

不保存模型密钥
不执行生成代码

BFF / Agent

认证、任务状态
组织模型上下文

模型提出方案

Intent、UI Schema
Tool Proposal

验证与确认

类型、权限、状态
用户明确确认

Tool Gateway

幂等、审计、补偿
调用业务服务

最小替代方案：模型只返回受 Schema 约束的建议；客户端只渲染白名单组件；数据库只允许业务服务通过可信工具访问。

模型可以建议下一步，但不能自行扩大权限、绕过确认或成为业务数据的直接操作者

Part 2

AI 原生软件开发环境全景

一套项目，包含五类开发环境



环境搭建的目标是让五类环境通过明确协议连接，而不是每个人电脑上安装一堆互不相干的软件。

开发机器与账号准备

本地机器

- Windows 或 macOS 开发环境
- 为 IDE、SDK、模拟器和容器预留充足磁盘
- 启用硬件虚拟化以运行模拟器和容器
- 保持稳定网络，准备代理和镜像源故障方案
- 真机开发时准备数据线和调试授权

课程账号

- 华为开发者账号
- 代码托管与团队协作账号
- 课程 LLM Gateway 访问凭证
- 镜像仓库或部署平台账号
- 不得共享个人密钥和签名材料

DevEco Studio、SDK 和模拟器要求会随版本变化，安装前以当学期官方文档和教师冻结版本为准。

DevEco Studio：鸿蒙端的主开发环境

工程与代码

创建 HarmonyOS 应用工程，编辑 ArkTS、资源和模块配置。

构建

通过 Hvigor 管理源码、资源、依赖和多目标构建。

预览

快速查看 ArkUI 页面和组件在不同设备形态下的效果。

调试

断点、变量、调用栈、日志和跨语言调试。

设备

管理模拟器、真机连接、安装、运行和签名。

性能

使用 Profiler 检查启动、网络、内存、耗时和卡顿。

HarmonyOS SDK 与项目版本必须统一

SDK 决定什么

- 可使用的 ArkTS、ArkUI 和 Kit API
- 编译、打包、签名和设备工具
- 模拟器镜像与目标系统能力
- 最低兼容版本和目标版本

团队统一什么

- DevEco Studio 主版本
- HarmonyOS SDK/API 版本
- 模拟器镜像与设备形态
- 依赖锁文件和构建配置
- 升级时间点和兼容策略

“我电脑上能运行”不是可复现环境；版本、配置和依赖必须能够被团队重新建立。

模拟器、预览器与真机各有用途

环境	适合	局限	课程要求
ArkUI Preview	快速调整组件和布局	不能代表完整应用运行	用于日常 UI 开发
本地模拟器	应用流程、调试、自动化测试	部分系统能力与真机不同	每组必须可运行
远程/共享设备	补充设备覆盖	网络、配额和可用时间	按资源条件使用
HarmonyOS 真机	真实性能、权限、网络和系统能力	设备与签名成本	重要里程碑至少验证一次

鸿蒙端第一次验收：Hello CampusHub

必须完成

1. 创建一个 Empty Ability 工程。
2. 使用 ArkTS 和 ArkUI 显示 CampusHub 首页。
3. 加入一个输入框和“发送”按钮。
4. 在模拟器或真机运行。
5. 设置断点并查看一次状态变化。
6. 发出一次 HTTP 请求并显示结果。
7. 提交可复现的构建和运行说明。

输入：帮我找一个今晚顺路取快递的人

输出：后端连接成功 / 当前使用 Mock Agent

验收标准：另一位同学根据 README 能够在自己的环境中构建并运行。

ArkTS 与 ArkUI 环境中要认识这些文件

应用与模块配置

应用信息、Module、Ability、权限、设备类型和构建目标。

页面与组件

ArkTS 页面、可复用组件、状态、资源和路由。

构建配置

Hvigor、依赖、产品配置和签名相关设置。

测试目录

单元测试、组件测试和应用测试应与源码共同维护。

不要把 AI 生成的所有代码堆进一个页面文件；从第一周就建立领域、应用、UI 和基础设施边界。

仓颉环境：语言实验与后端组件

核心工具

cjc

仓颉编译命令，将源码编译为可执行产物

CJPM

官方项目管理与统一构建入口

cjfmt

代码格式化工具

推荐开发方式

- 安装与平台架构匹配的仓颉 SDK
- 安装 VS Code 与仓颉语言插件
- 配置 SDK 绝对路径或执行环境脚本
- 验证 `cjc`、`cjpm` 和 `cjfmt`
- 创建并构建第一个 CJPM 项目

仓颉环境验收

```
main() {  
    println("CampusHub Cangjie service ready")  
}
```

```
cjc hello.cj -o hello  
./hello
```

```
# 或在 CJPM 项目中  
cjpm build  
cjpm run
```

验收证据

- SDK 与目标平台架构匹配。
- 命令行可以找到编译器和包管理器。
- IDE 能够完成语法检查、跳转和调试。
- 项目能够从干净目录重新构建。

Part 3

智能体后端、模型与数据环境

三版演示环境展示一条渐进式工程路线

版本	模型与交互	业务与数据	主要教学目标
V1: 最小闭环	Mock 模型、规则抽取、动态确认卡	零依赖 Java、内存状态	看懂 Agent、Tool 与人在回路
V2: 真实模型	OpenAI 兼容接口、结构化输出校验	沿用两阶段确认与幂等	理解模型不可信与失败降级
V3: 独立全栈	Spring Boot Agent API、MCP JSON-RPC	JWT、JDBC、H2/PostgreSQL、审计	连接 AI 能力与真实软件工程

先跑通

无密钥也能演示
协议与确认



再替换

接入真实模型
验证不确定输出



最后工程化

认证、数据库
审计与标准工具

三个项目相互独立，便于比较每增加一层能力，也增加了哪些验证责任

本地基础工具：负责构建、启动与诊断

工具	在演示中的作用	学生需要掌握
JDK	编译和运行 Java/Spring Boot 业务服务	<code>java -version</code> 、 <code>javac</code> 、LTS 版本统一
Maven	解析 Spring 依赖、编译、测试和启动	<code>mvn test</code> 、 <code>mvn spring-boot:run</code>
Node.js	V1/V2 Agent Gateway、浏览器页面和端到端测试	<code>node --test</code> 、环境变量、异步 HTTP
Make	把多条命令封装为统一课堂入口	<code>make doctor</code> 、 <code>make start</code> 、 <code>make status</code>
curl	绕开 UI 直接验证 Health、登录、MCP 和业务 API	请求头、JSON Body、状态码与错误正文
lsof	定位端口占用与重复启动	<code>lsof -nP -iTCP:4200 -sTCP:LISTEN</code>

IDE 用于提高开发效率；命令行用于建立可复现、可自动化、可诊断的工程入口。

Java 参考实现中的框架与基础设施

Spring Boot

统一 HTTP API、静态页面、配置、依赖注入、异常处理与应用生命周期。

Spring Web / RestClient

提供登录、对话、确认、MCP 端点，并调用 OpenAI 兼容模型接口。

Spring JDBC

用确定性 SQL 保存需求与审计事件，让数据库约束守住业务底线。

H2

课堂默认的文件型数据库，无需 Docker；重启后仍可查看已发布需求。

PostgreSQL

生产 Profile 的关系数据库，承担事务、唯一约束和长期业务事实。

Redis

生产环境保存短期确认令牌、幂等状态、限流和任务进度，不代替业务数据库。

身份、安全与数据一致性工具

JWT

登录后由服务器签发身份令牌；Tool Gateway 从已验证令牌注入用户身份，模型不能自行填写 `userId`。

确认令牌

`prepare_publish` 生成短期一次性令牌，把“模型建议”与“真实发布”分开。

业务本体规范化

把“快递服务”等模型近义词映射为“快递代取”，无法映射时归入“其他”并记录审计。

JSON Schema / 白名单

限定模型可以生成的字段、分类和 UI 类型；未知组件不能进入客户端渲染。

幂等键与唯一约束

重复点击或网络重试只产生一条需求；按钮禁用不是唯一的防重复机制。

Trace 与审计表

记录模型完成、分类规范化、工具准备、确认发布和最终业务编号。

模型环境与工具协议各自解决不同问题

模型侧

OpenAI 兼容 API

以统一的 `/v1/chat/completions` 契约连接不同模型

LiteLLM

可选 Model Gateway：路由、虚拟密钥、配额、成本、重试与降级

Ollama

可选本地模型运行时，用于离线、隐私和小模型对照实验

工具侧

MCP / JSON-RPC

标准化工具发现与调用；V3 提供 `tools/list` 和 `tools/call`

Tool Gateway

在协议之外增加身份、权限、白名单、确认、幂等和审计

Spring AI MCP

后续可将 Java 方法以标准 MCP Server 暴露给不同 Agent 客户端

Model Gateway 管“调用哪个模型”；MCP/Tool Gateway 管“模型可以请求哪些真实动作”。两者不能互相替代。

客户端、协议与动态 UI 工具链

浏览器演示客户端

低成本验证对话、临时卡片、确认按钮和完成态；与 HarmonyOS 共用 HTTP 契约。

DevEco Studio

将浏览器替身替换为 ArkTS/ArkUI 客户端，并完成模拟器、真机、签名与性能调试。

UI Schema Renderer

只渲染白名单组件；卡片按当前任务生成，完成后进入完成态或被丢弃。

普通 HTTP

适合登录、短模型调用、确认发布、需求列表和健康检查。

SSE

后续用于模型增量文本、工具进度与任务状态，保持服务器到客户端的单向流。

WebSocket

只在高频双向消息和即时通讯场景使用，不作为所有接口的默认选择。

后端可以保留 Java，并按职责组合技术栈

稳定业务后台：Java

Spring Boot 保留用户、需求、订单、信用、权限和事务等已有业务能力，通过 API 或 Tool Gateway 向智能体开放。

智能体服务：Python

FastAPI + Pydantic + uv，适合 Agent 编排、模型 SDK、RAG、评测和数据处理。

可选服务：TypeScript

Node.js + TypeScript，适合共享 Schema、Web 生态、SSE/WebSocket 和事件驱动服务。

专项组件：仓颉

用于类型安全、并发和高性能工具或事件组件，不要求重写整个后端。

推荐演化路径：保留 Java 业务核心，在其前面增加智能体编排层；不要为了接入 LLM 重写已经可靠运行的业务后台。

Java Web 后端环境

适合继续承担

- 用户、身份、权限和组织关系
- 需求、订单、评价和信用状态机
- 数据库事务、幂等和一致性
- 内容审核、管理后台和统计接口
- 面向智能体的可信业务工具

推荐工程环境

- 统一 JDK LTS 版本
- Spring Boot 与 Maven 或 Gradle
- JUnit、MockMvc/Testcontainers
- Flyway 或 Liquibase 数据库迁移
- OpenAPI、结构化日志和健康检查

```
./mvnw test
./mvnw spring-boot:run

# 或
./gradlew test
./gradlew bootRun
```

Agent 不直接访问 Java 服务的数据库，而是调用经过鉴权、参数校验和审计的业务 API/Tool。

Python 后端环境

推荐组件

- Python 稳定版本
- uv: 版本、虚拟环境和依赖管理
- FastAPI: HTTP 与流式 API
- Pydantic: 请求、事件与 Schema 校验
- pytest: 单元与集成测试
- Ruff / 类型检查: 静态质量门禁

```
uv sync
uv run pytest
uv run fastapi dev app/main.py
```

环境原则

- 不向系统 Python 全局安装课程依赖
- 提交锁文件, 不只提交依赖名称
- 开发、测试和生产使用相同配置结构
- 启动命令写入 README 或任务脚本
- 在 CI 中从零创建环境并运行测试

| Node.js / TypeScript 后端环境

适用场景

- 前后端共享 JSON Schema 和类型
- SSE、WebSocket 和事件驱动接口
- 大量 Web 与模型 SDK 集成
- 团队已有 TypeScript 经验

```
pnpm install --frozen-lockfile  
pnpm test  
pnpm dev
```

环境原则

- 统一 Node.js LTS 版本
- 固定一个包管理器
- 提交 lockfile
- 启用严格 TypeScript、lint 和测试
- 禁止把密钥写入前端或仓库

数据环境：业务事实、任务状态与短期缓存分开

PostgreSQL

用户、需求、订单、授权、事件索引等需要事务和长期保存的业务事实。

Redis

短期会话、限流、缓存、幂等键、流式任务进度和临时状态。

对象/文件存储

图片、附件、评测集、日志归档和较大模型输入输出。

不要把所有东西放进对话历史

订单状态、权限和已确认事实必须进入正式数据模型。

不要把 Redis 当永久数据库

缓存和临时任务失效后，核心业务事实仍应可恢复。

容器环境：统一依赖服务，不容器化一切

适合放入容器

- PostgreSQL、Redis 等共享依赖
- 参考后端与工具服务
- 观测、测试和数据处理组件
- CI 中需要重复建立的环境

```
docker compose up -d postgres redis
docker compose ps
docker compose logs --tail=50
```

本地直接运行更方便

- DevEco Studio 与 HarmonyOS 模拟器
- 需要频繁断点调试的客户端
- 仓颉 SDK 和本地语言工具链
- 需要硬件加速的本地模型需单独评估

LLM 环境一：通过课程 Model Gateway 接入云模型



LLM 环境二：本地或私有模型用于可选实验

适用目标

隐私敏感处理、离线实验、成本比较、模型路由和降级验证。

资源约束

显存/内存、模型大小、量化方式、上下文长度、Token/s 和并发。

课程边界

不要求每组训练或部署大型模型；可使用教师共享服务或小模型。

需要记录

- 模型名称与版本
- 运行硬件与内存
- 启动参数与上下文限制
- 吞吐、延迟和失败率

需要比较

- 任务质量
- 数据是否出域
- 单次推理成本
- 故障后的替代路径

Model Gateway 统一模型差异

接口统一

对上层提供稳定的消息、结构化输出、流式响应和错误语义。

路由选择

根据任务、隐私、质量、延迟、成本和可用性选择模型。

配额与成本

按用户、团队和任务限制请求、Token 和预算。

可靠性

超时、重试、熔断、备用模型和固定流程降级。

版本管理

记录模型、Prompt、参数和输出 Schema 版本。

审计与脱敏

记录必要元数据，同时避免敏感内容进入普通日志。

Tool Gateway 把模型与真实业务隔离

一个可信工具契约包含

- 明确名称与业务语义
- 结构化输入输出 Schema
- 调用者身份与资源级权限
- 允许调用的业务状态
- 是否需要用户确认
- 幂等、错误、超时和补偿语义

演示项目中的真实工具

`demand.prepare_publish`

校验草稿，生成短期确认令牌，不写业务表

`demand.confirm_publish`

验证身份、确认令牌和幂等键后写入需求

`tools/list`

通过 MCP 发现允许调用的工具和输入契约

流式接口环境：SSE 优先，WebSocket 按需

方式	特点	适合 AI Native
普通 HTTP	一次请求、一次响应	登录、普通 CRUD、短工具调用
SSE	服务端持续向客户端推送，基于 HTTP	模型文本、UI Schema、工具进度和任务状态
WebSocket	双向长连接	高频双向事件、即时通讯和协同场景

`assistant.delta` → `ui.schema` → `tool.pending` → `tool.result` → `task.state`

必须处理

断线、重连、消息顺序、重复消息、超时和任务恢复。

必须观测

首 Token 延迟、总耗时、流式中断率和工具等待时间。

可观测环境：一次请求必须能够完整追踪



测试环境从第一天进入项目

单元测试

领域规则、Schema 校验、权限判断和数据转换。

契约测试

客户端、Agent、模型和工具之间的消息与错误语义。

集成测试

后端、数据库、缓存、模型 Mock 和工具服务。

端到端测试

从鸿蒙客户端输入到业务状态和最终界面。

AI 评测

固定任务集、结构化输出、失败分类、质量、延迟和成本。

红队测试

提示注入、越权、恶意 Schema、工具滥用和隐私泄露。

Part 4

工程协作、配置与环境验收

| Git 仓库是团队和 AI 的共同工作区

仓库必须包含

- 客户端、后端、工具与共享 Schema
- README 与一键启动说明
- 版本文件、依赖锁文件
- 测试、评测样本和环境健康检查
- 架构决策记录与变更说明

协作规则

- 小分支、小提交、小范围 AI 任务
- 合并前构建、测试和人工评审
- 不覆盖其他成员未提交的工作
- AI 生成内容必须由责任人署名验收
- 关键协议变更必须更新版本和 ADR

推荐的项目目录

```
campushub-ai/
├── apps/
│   └── harmony/           # ArkTS / ArkUI 客户端
├── services/
│   ├── agent/            # Agent Orchestrator / BFF
│   ├── tools/            # 可信工具服务
│   └── cangjie/          # 仓颉实验或专项服务
├── packages/
│   ├── schemas/          # UI、事件、工具共享 Schema
│   └── ontology/         # 业务本体与版本
├── tests/
│   ├── contract/
│   ├── e2e/
│   └── evals/
├── deploy/               # Compose、部署与观测配置
├── docs/                 # ADR、数据字典、运行手册
├── .env.example
└── README.md
```

目录可以调整，但稳定协议、业务知识和测试不应被埋在某个应用内部

配置与密钥：可以复制结构，不能复制秘密

.env.example

只列变量名称、用途和安全示例，不包含真实密钥、密码或个人地址。

本地 .env

由开发者自行创建，加入忽略列表，并限制文件权限。

CI / 部署 Secrets

通过平台的秘密管理功能注入，不写入脚本、镜像或日志。

密钥泄露处理

立即撤销、轮换、检查日志和提交历史，不能只删除当前文件。

```
MODEL_GATEWAY_URL=https://gateway.example.edu
MODEL_API_KEY=replace_me
DATABASE_URL=postgresql://user:password@localhost:5432/campusdb
REDIS_URL=redis://localhost:6379/0
```

环境配置要能被机器检查

建议提供统一诊断命令

```
task doctor
```

检查内容

版本

DevEco/SDK、Python/Node、仓颉、Docker 和数据库版本。

端口

后端、PostgreSQL、Redis 和观测服务端口是否可用。

依赖

包安装、锁文件、数据库迁移和容器健康状况。

凭证

只检查是否配置与是否有效，不打印真实内容。

连通

客户端 API、Model Gateway 和工具健康检查。

最小测试

运行一个模型 Mock 和一个工具调用闭环。

常见环境故障与定位顺序

现象	先检查	再检查
鸿蒙工程无法构建	IDE/SDK/API 版本与依赖锁	缓存、构建配置和签名
模拟器无法运行	虚拟化、镜像、磁盘和端口	hdc 连接与系统权限
客户端连不上后端	URL、端口、网络权限和日志	代理、证书、CORS/网关配置
模型调用失败	Gateway 健康、凭证、配额	模型名、超时、结构化输出
数据库连接失败	容器健康、端口、连接串	迁移、账号权限和数据卷
仓颉命令找不到	SDK 路径和环境脚本	平台架构、IDE 插件设置

定位顺序：复现 → 读取错误 → 检查边界 → 验证假设；不要一上来就删除缓存或重装全部环境。

第一章环境验收任务

团队必须现场演示

1. HarmonyOS 客户端在模拟器或真机运行。
2. 用户输入一条 CampusHub 目标。
3. 后端通过 SSE 返回文本片段和一张 Mock 卡片 Schema。
4. 客户端验证并渲染卡片。
5. 用户点击确认，调用一个 Mock Tool。
6. Tool 写入一条测试业务记录或事件。
7. 日志中使用同一个 `trace_id` 串联整个调用链。
8. 模型服务关闭后，系统退回 Mock Agent 或固定响应。

另一组从干净环境按照 README 复现成功，才算通过环境验收。

环境验收评分

维度	比例	通过标准
鸿蒙端环境	20%	可构建、运行、调试和发起网络请求
后端与数据	20%	API、数据库、缓存和健康检查正常
模型与工具	20%	Gateway 与 Mock Tool 均可调用，并有降级
端到端链路	20%	文本、Schema、确认、工具和事件完整运行
可复现性	10%	版本、锁文件、README 和诊断命令完整
安全与协作	10%	无密钥泄露；提交、评审和责任人清晰

环境验收是后续所有里程碑的前置条件，不单独追求复杂功能

第一周最小工作清单

鸿蒙端

DevEco、SDK、模拟器/真机、Empty Ability、网络请求。

仓颉

SDK、VS Code 插件、cjc、CJPM 和 Hello World。

后端

参考运行时、依赖锁、API、SSE 和测试。

数据

PostgreSQL、Redis、迁移和健康检查。

模型

课程 Gateway、Mock Agent、凭证和降级路径。

工程

Git、README、.env.example、CI 和统一诊断命令。

先建立可以运行、可以验证、可以复现的最小系统，再让 AI 加速功能开发。

官方资料入口

HarmonyOS 开发

DevEco Studio、HarmonyOS SDK、ArkTS、ArkUI、模拟器、调试与性能工具。

仓颉语言

SDK 安装、VS Code 插件、cjc、CJPM、cjfmt、语言指南和工具链。

容器与数据

Docker、PostgreSQL、Redis 的官方安装、健康检查和数据持久化文档。

可观测性

OpenTelemetry 的日志、指标、Trace 和语言 SDK 入门文档。

课程正式开始前以教师发布的版本矩阵为准；不要自行升级某一个关键组件后直接提交到主分支。

本章知识地图



先让完整链路跑起来

下一章：个性化需求与校园业务本体——让 AI 知道每个人想要什么，也理解校园互助意味着什么